الاحتراف الكامل

في مكتبات بايثون لعلم البيانات والتعلّم الآلي

الجزء الثاني: مكتبة NumPy

الفصول ٧ - ١١: الأمثلة العملية، الأداء، التصحيح، أفضل الممارسات، والمشاريع المصغّرة

مرجع احترافي للمبرمجين والمهندسين وعلماء البيانات

جدول المحتويات

جدول المحتويات 2
مقدمة الجزء الثاني 4
الفصل السابع: الأمثلة العملية 5
٧.١ المستوى المبتدئ 5
مثال ١: حساب معدل درجات طلاب 5
مثال ٢: تطبيع بيانات (Min-Max Normalization) 5
٧.٢ المستوى المتوسط 5
مثال ٣: معالجة صورة كمصفوفة NumPy 5
مثال ٤: تجميع بيانات مبيعات حسب الفئة (بدون Pandas) 5
٧.٣ المستوى الاحترافي 6
مثال ٥: حساب المسافة الإقليدية بين مجموعتي نقاط (Vectorized) 6
مثال ٦: نافذة متحركة (Rolling Window) بدون مكتبات خارجية 6
٧.٤ مثال من مشروع حقيقي: خط أنابيب معالجة بيانات استشعار 6
تمارين عملية 7
حلول مقترحة (تلميحات) 7
الفصل الثامن: الأداء (Performance) 8
٨.١ لماذا يهتم المحترف بالأداء في NumPy تحديدًا 8
٨.٢ استهلاك الذاكرة (Memory Usage) 8
٨.٣ Copy مقابل View: الأثر على الأداء 8
٨.٤ Vectorization والقياس الفعلي للفرق 8
٨.٥ أدوات القياس (Profiling) 9
٨.٦ اختناقات الأداء الشائعة 9
الأخطاء الشائعة 9
أفضل الممارسات 10
الفصل التاسع: التصحيح (Debugging) 11
٩.١ قراءة رسائل الخطأ الشائعة 11
٩.٢ استراتيجية تشخيص أخطاء Shape 11
٩.٣ التحذيرات (Warnings) وكيفية التعامل معها 11
٩.٤ أدوات تشخيصية مفيدة 11
٩.٥ استخدام Logging بدل print في الكود الاحترافي 12
الأخطاء الشائعة 12
الأسئلة الشائعة 12
الفصل العاشر: أفضل الممارسات (Best Practices) 13

١٠.١ أسلوب الكتابة الاحترافي 13
١٠.٢ قابلية القراءة والصيانة 13
١٠.٣ هيكلة المشروع 13
١٠.٤ الاختبار (Testing) 13
١٠.٥ التوثيق (Documentation) 14
أفضل الممارسات 14
الفصل الحادي عشر: مشاريع مصغّرة 15
مشروع ١: محاكاة نرد وحساب احتمالات تجريبية 15
مشروع ٢: ضغط صورة رمادية بسيط عبر تقليل الدقة (Downsampling) 15
مشروع ٣: محرك توصية بسيط بالتشابه الإقليدي 15
تمارين عملية 16
حلول مقترحة (تلميحات) 16
مراجع وروابط رسمية 16
خاتمة NumPy 16

مقدمة الجزء الثاني

يبني هذا الجزء مباشرة على الجزء الأول، الذي أرسى الأساس النظري لمكتبة NumPy: تاريخها، فلسفة تصميمها، ومفاهيمها الجوهرية مثل Shape وStrides وViews وBroadcasting، إضافة إلى أهم دوالها. في هذا الجزء ننتقل من الفهم إلى الاحتراف العملي: أمثلة متدرجة من مشاريع حقيقية، ثم فهم عميق للأداء واستهلاك الذاكرة، فطرق التصحيح والتعامل مع الأخطاء، ثم أفضل الممارسات المتبعة في الشركات الاحترافية، وأخيرًا مشاريع مصغّرة كاملة تطبّق كل ما سبق دون أي تدريب لنماذج تعلّم آلي.

الفصل السابع: الأمثلة العملية

هذا الفصل يتدرّج من أمثلة بسيطة جدًا إلى أمثلة احترافية مستوحاة من مشاريع حقيقية، مع شرح كل سطر من الكود.

٧.١ المستوى المبتدئ

مثال ١: حساب معدل درجات طلاب

```
import numpy as np

grades = np.array([88, 92, 79, 95, 61, 74])
average = grades.mean()           # حساب المتوسط الحسابي مباشرة
above_avg = grades[grades > average]   # فهرسة منطقية: من هم فوق المتوسط

print(f"المتوسط: {average:.2f}")
print(f"فوق المتوسط: {above_avg}")
```

شرح الكود: نبني مصفوفة أحادية البُعد من الدرجات، ثم نستخدم .mean() لحساب المتوسط عبر عملية مجمَّعة سريعة، ثم نستخدم Boolean Indexing لتصفية القيم الأعلى من المتوسط في سطر واحد دون أي حلقة for صريحة.

مثال ٢: تطبيع بيانات (Min-Max Normalization)

```
prices = np.array([120.0, 340.5, 89.9, 560.0, 210.0])
normalized = (prices - prices.min()) / (prices.max() - prices.min())
print(normalized)   # كل القيم أصبحت بين 0 و1
```

شرح الكود: هذا مثال نموذجي على Vectorization: العملية الحسابية بأكملها (طرح، ثم قسمة) تُطبَّق على المصفوفة كلها دفعة واحدة بفضل Broadcasting، دون كتابة أي حلقة يدوية.

٧.٢ المستوى المتوسط

مثال ٣: معالجة صورة كمصفوفة NumPy

```
# افتراض أن image 3 مصفوفةD بشكل (height, width, channels)
image = np.random.randint(0, 256, (100, 100, 3), dtype=np.uint8)

gray = image.mean(axis=2).astype(np.uint8)   # تحويل تقريبي لتدرج الرمادي
flipped = image[:, ::-1, :]                  # قلب أفقي عبر Slicing بخطوة سالبة
cropped = image[20:80, 20:80, :]             # قص جزء من الصورة
```

شرح الكود: axis=2 يعني حساب المتوسط عبر القنوات اللونية الثلاث (R,G,B) لكل بكسل، مما يحوّلها إلى قيمة رمادية واحدة. القلب الأفقي والقص كلاهما ينتجان Views وليس نسخًا، وهذا يوضّح كفاءة NumPy في معالجة الصور.

مثال ٤: تجميع بيانات مبيعات حسب الفئة (بدون Pandas)

```
categories = np.array(['A', 'B', 'A', 'C', 'B', 'A'])
sales = np.array([100, 200, 150, 300, 250, 120])

unique_cats = np.unique(categories)
totals = {cat: sales[categories == cat].sum() for cat in unique_cats}
print(totals)   # {'A': 370, 'B': 450, 'C': 300}
```

شرح الكود: np.unique تستخرج الفئات الفريدة، ثم لكل فئة نستخدم Boolean Indexing لاستخراج المبيعات المطابقة وجمعها. هذا المثال يوضّح أيضًا لماذا Pandas أنسب لهذا النوع من المهام في المشاريع الكبيرة، فهي توفّر groupby جاهزة وأكثر كفاءة.

٧.٣ المستوى الاحترافي

مثال ٥: حساب المسافة الإقليدية بين مجموعتي نقاط (Vectorized)

```
points_a = np.random.rand(1000, 2)   # 1000 نقطة بإحداثيتين
points_b = np.random.rand(1000, 2)

# الطريقة البطيئة (حلقة صريحة)
distances_loop = np.array([
    np.sqrt(((a - b) ** 2).sum())
    for a, b in zip(points_a, points_b)
])

# الطريقة المتجهة (Vectorized) – أسرع بعشرات المرات
diff = points_a - points_b
distances_vectorized = np.sqrt((diff ** 2).sum(axis=1))
```

شرح الكود: في النسخة المتجهة، الطرح يتم بين مصفوفتين كاملتين دفعة واحدة، ثم التربيع، ثم الجمع عبر axis=1 (أي جمع إحداثيات كل نقطة)، ثم الجذر التربيعي. هذه السلسلة الكاملة تُنفَّذ في شيفرة C دون أي حلقة بايثون، وهو الفرق الجوهري بين الكود المبتدئ والكود الاحترافي في NumPy.

مثال ٦: نافذة متحركة (Rolling Window) بدون مكتبات خارجية

```
from numpy.lib.stride_tricks import sliding_window_view

series = np.array([1, 3, 5, 7, 9, 11, 13])
windows = sliding_window_view(series, window_shape=3)
moving_avg = windows.mean(axis=1)
print(moving_avg)   # [3. 5. 7. 9. 11.]
```

شرح الكود: sliding_window_view تنشئ Views متعددة الأبعاد فوق نفس الذاكرة الأصلية دون أي نسخ، مستغلة مفهوم Strides الذي شرحناه في الفصل الرابع، وهي مثال ممتاز على الاستخدام المتقدم لفهم البنية الداخلية للمصفوفة.

٧.٤ مثال من مشروع حقيقي: خط أنابيب معالجة بيانات استشعار

```
# بيانات وهمية من مستشعر حرارة (قراءة كل ثانية لمدة دقيقة)
readings = np.random.normal(loc=22.0, scale=1.5, size=60)
readings[10:12] = np.nan   # محاكاة قيم مفقودة من عطل مؤقت في المستشعر

valid_mask = ~np.isnan(readings)        # قناع القيم الصالحة
clean = readings[valid_mask]            # تصفية القيم المفقودة

mean_temp = clean.mean()
std_temp = clean.std()
outliers = clean[np.abs(clean - mean_temp) > 2 * std_temp]  # قيم شاذة (Outliers)
```

```
print(f"القراءات الصالحة: {clean.size} من {readings.size}")
print(f"المتوسط: {mean_temp:.2f} | الانحراف المعياري: {std_temp:.2f}")
print(f"القيم الشاذة: {outliers}")
```

شرح الكود: np.isnan تكشف القيم المفقودة (NaN)، وعامل ~ يعكس القناع المنطقي لاستخراج القيم الصالحة فقط. تحديد القيم الشاذة هنا يعتمد على قاعدة إحصائية بسيطة (خارج ضعف الانحراف المعياري)، وهذا مثال نموذجي لخط أنابيب تنظيف بيانات كامل باستخدام NumPy وحدها قبل تمرير البيانات لمكتبات أعلى مستوى مثل Pandas أو Scikit-learn.

تمارين عملية

١. بيانات حرارة يومية على مدار أسبوع (٧×٢٤ قراءة). أنشئ مصفوفة بشكل (٧, ٢٤) واحسب متوسط كل يوم ومتوسط كل ساعة عبر كل الأيام.

٢. أنشئ مصفوفة درجات طلاب بشكل (٣٠ طالبًا × ٥ مواد)، واحسب معدل كل طالب، ثم استخرج الطلاب الحاصلين على معدل أعلى من ٨٥.

٣. استخدم sliding_window_view لحساب المتوسط المتحرك بحجم نافذة ٥ على سلسلة من ١٠٠ رقم عشوائي.

حلول مقترحة (تلميحات)

١. استخدم reshape(7, 24)، ثم mean(axis=1) لليوم و mean(axis=0) للساعة.

٢. استخدم mean(axis=1) لحساب معدل كل طالب (كل صف)، ثم Boolean Indexing للتصفية.

٣. sliding_window_view(data, 5).mean(axis=1).

الفصل الثامن: الأداء (Performance)

٨.١ لماذا يهتم المحترف بالأداء في NumPy تحديدًا

الهدف الأساسي من استخدام NumPy أصلاً هو الأداء؛ فإذا كتبت كودًا يستخدم NumPy لكنه لا يستغل Vectorization، فأنت تتحمّل تعقيد المكتبة دون الاستفادة من ميزتها الحقيقية. هذا الفصل يشرح كيف تقيس الأداء، وأين تكمن اختناقاته الشائعة، وكيف تحسّنه بثقة بدل التخمين.

٨.٢ استهلاك الذاكرة (Memory Usage)

```
a = np.zeros((1000, 1000), dtype=np.float64)
print(a.nbytes / 1e6, 'MB')     # 8.0 MB

b = a.astype(np.float32)
print(b.nbytes / 1e6, 'MB')     # 4.0 MB — نصف الاستهلاك
```

اختيار dtype المناسب هو أول وأبسط قرار لتحسين استهلاك الذاكرة. في مشاريع تعلّم الآلة التي تتعامل مع ملايين الصفوف، الفرق بين float64 وfloat32 قد يعني الفرق بين تشغيل الكود بنجاح أو نفاد الذاكرة (Out of Memory).

٨.٣ Copy مقابل View: الأثر على الأداء

كل عملية تُنتج Copy تستهلك وقتًا لتخصيص ذاكرة جديدة ونسخ البيانات إليها. في الحلقات الحرجة للأداء، كثرة النسخ غير الضرورية سبب شائع لبطء غير متوقّع. من المفيد معرفة أي العمليات تُنتج Views وأيها تُنتج Copies دائمًا.

العملية	النتيجة
Basic Slicing (a[1:5])	View
Fancy Indexing (a[[1,3,5]])	Copy
Boolean Indexing (a[a>0])	Copy
reshape (عند التوافق)	View
transpose / .T	View
()flatten	Copy دائمًا
()ravel	View إن أمكن، وإلا Copy

٨.٤ Vectorization والقياس الفعلي للفرق

```
import time

n = 2_000_000
data = np.random.rand(n)

start = time.perf_counter()
result_loop = [x * 2 + 1 for x in data]     # حلقة بايثون عادية
print('loop:', time.perf_counter() - start)
```

```
start = time.perf_counter()
result_vec = data * 2 + 1                    # عملية متجهة
print('vectorized:', time.perf_counter() - start)
# الفرق عادة بين 20 و 100 ضعف لصالح النسخة المتجهة
```

٨.٥ أدوات القياس (Profiling)

```
# قياس زمن سريع داخل Jupyter
%timeit data * 2 + 1

# قياس تفصيلي لسطر بسطر (يتطلب: pip install line_profiler)
%load_ext line_profiler
%lprun -f my_function my_function(data)

# قياس استهلاك الذاكرة (يتطلب: pip install memory_profiler)
%load_ext memory_profiler
%memit my_function(data)
```

%timeit هو الأداة الأسرع والأكثر استخدامًا يوميًا لأنها تكرر القياس تلقائيًا وتستبعد تأثير التذبذبات العشوائية في زمن التنفيذ.

٨.٦ اختناقات الأداء الشائعة

الحل	الاختناق
أعد كتابة العملية كتعبير مصفوفي كامل	استخدام حلقة for بدل عملية متجهة
خصّص المصفوفة بحجمها النهائي مسبقًا بـ np.empty ثم املأها	استخدام np.append داخل حلقة لبناء مصفوفة تدريجيًا
استخدم Views (Basic Slicing) حيثما أمكن	نسخ غير ضرورية عبر .copy() أو Fancy Indexing المتكرر
حدّد dtype عددي صريح مثل float64 أو int32	استخدام dtype=object لبيانات عددية
نفّذ التحويل مرة واحدة خارج الحلقة	تحويل DataFrame إلى NumPy وبالعكس بشكل متكرر داخل حلقة

```
# سيئ الأداء: append داخل حلقة
result = np.array([])
for x in range(10000):
    result = np.append(result, x ** 2)   # ينسخ المصفوفة بالكامل في كل تكرار!

# جيد: تخصيص مسبق
result = np.empty(10000)
for i in range(10000):
    result[i] = i ** 2

# الأفضل: Vectorization كامل
result = np.arange(10000) ** 2
```

الأخطاء الشائعة

- استخدام np.append أو np.concatenate داخل حلقة بدل تخصيص المصفوفة مرة واحدة.

- نسيان أن Fancy Indexing وBoolean Indexing تنتج نسخًا، مما قد يضاعف استهلاك الذاكرة دون انتباه في البيانات الكبيرة.
- قياس الأداء بتشغيل واحد فقط بدل استخدام %timeit الذي يكرر القياس ويعطي نتيجة موثوقة إحصائيًا.

أفضل الممارسات

- خصّص المصفوفات بحجمها النهائي مسبقًا بدل توسيعها تدريجيًا.
- استخدم أصغر dtype يفي بالغرض دون فقدان دقة مؤثرة.
- قِس قبل أن تحسّن: استخدم %timeit وline_profiler لتحديد الاختناق الحقيقي بدل التخمين.
- فضّل التعبيرات المتجهة الكاملة (a * 2 + 1) على أي حلقة for صريحة.

الفصل التاسع: التصحيح (Debugging)

٩.١ قراءة رسائل الخطأ الشائعة

رسالة الخطأ	السبب الشائع
ValueError: operands could not be broadcast together	محاولة عملية بين مصفوفتين بأشكال غير متوافقة وفق قواعد Broadcasting
IndexError: index N is out of bounds	محاولة الوصول لفهرس خارج حدود المصفوفة
ValueError: the truth value of an array is ambiguous	استخدام مصفوفة متعددة العناصر مباشرة في شرط if
TypeError: ufunc did not contain a loop with signature matching types	محاولة عملية رياضية بين أنواع بيانات غير متوافقة (مثل نصوص وأرقام)
RuntimeWarning: invalid value encountered	عملية أنتجت NaN، غالبًا بسبب قسمة ٠/٠

٩.٢ استراتيجية تشخيص أخطاء Shape

أكثر أخطاء NumPy شيوعًا لدى المبرمجين من كل المستويات هي أخطاء عدم توافق الأشكال. الاستراتيجية العملية هي طباعة .shape لكل مصفوفة قبل العملية المشبوهة مباشرة، بدل محاولة تخمين الخطأ من رسالة النص فقط.

```
print('a.shape:', a.shape)
print('b.shape:', b.shape)
result = a + b   # الآن السبب واضح فورًا من الطباعة أعلاه
```

٩.٣ التحذيرات (Warnings) وكيفية التعامل معها

NumPy تصدر تحذيرات (لا أخطاء تُوقف البرنامج) في حالات مثل القسمة على صفر أو نتائج NaN. تجاهل هذه التحذيرات في بيانات حقيقية خطر لأنها غالبًا مؤشر على مشكلة حقيقية في البيانات.

```
import warnings

# تحويل التحذيرات إلى أخطاء فعلية أثناء التطوير للكشف المبكر
warnings.filterwarnings('error', category=RuntimeWarning)

# أو تسجيلها بدل تجاهلها
np.seterr(divide='warn', invalid='warn')
```

٩.٤ أدوات تشخيصية مفيدة

الأداة	الاستخدام
array.shape / .dtype / .ndim	فحص سريع لهيكل المصفوفة
np.shares_memory(a, b)	التحقق من مشاركة الذاكرة بين مصفوفتين
()np.isnan(a).sum	عدّ القيم المفقودة (NaN) في المصفوفة
()np.isfinite(a).all	التحقق من عدم وجود قيم لا نهائية أو NaN

الاستخدام	الأداة
فرض توقعات صريحة داخل الكود لالتقاط الأخطاء مبكرًا	assert a.shape == expected_shape

٩.٥ استخدام Logging بدل print في الكود الاحترافي

```
import logging
logging.basicConfig(level=logging.DEBUG)
logger = logging.getLogger(__name__)

def normalize(data: np.ndarray) -> np.ndarray:
    logger.debug('input shape=%s dtype=%s', data.shape, data.dtype)
    result = (data - data.mean()) / data.std()
    logger.debug('output range=[%s, %s]', result.min(), result.max())
    return result
```

الاعتماد على print() مقبول في الاستكشاف السريع داخل Jupyter، لكن في كود إنتاجي حقيقي يُفضَّل استخدام وحدة logging، لأنها تتيح التحكم بمستوى التفصيل (DEBUG, INFO, WARNING) دون تعديل الكود، وتوجيه المخرجات لملفات بدل الطرفية.

الأخطاء الشائعة

- استخدام if array بدل np.any() أو np.all() أو np.array_equal().
- تجاهل RuntimeWarning المتعلقة بـ NaN بدل معالجة سبب ظهورها الحقيقي في البيانات.
- الاعتماد فقط على رسالة الخطأ النصية دون طباعة .shape للمصفوفات المعنية.

الأسئلة الشائعة

س: كيف أعرف أي مصفوفة تحديدًا سبّبت خطأ Broadcasting في تعبير طويل؟

ج: قسّم التعبير الطويل إلى متغيرات وسيطة مسمّاة واطبع .shape لكل واحد منها بشكل منفصل قبل التجميع النهائي.

س: هل يجب إيقاف كل التحذيرات لتنظيف المخرجات؟

ج: لا يُنصح بذلك؛ فالتحذيرات مثل RuntimeWarning غالبًا تكشف مشكلة حقيقية في البيانات مثل قيم NaN أو قسمة على صفر.

الفصل العاشر: أفضل الممارسات (Best Practices)

١٠.١ أسلوب الكتابة الاحترافي

- استخدم أسماء متغيرات واضحة تصف المحتوى (prices بدل a) إلا في سياقات رياضية بحتة مختصرة ومفهومة محليًا.
  - التزم بمعيار PEP 8 لتنسيق الكود، ويمكن أتمتته عبر أدوات مثل black وruff.
  - أضف Type Hints عند تعريف الدوال التي تتعامل مع ndarray لتوضيح التوقعات.
  - افصل منطق المعالجة العددية عن منطق القراءة/الكتابة والعرض المرئي في وحدات (Modules) منفصلة.

```
import numpy as np

def z_score(data: np.ndarray) -> np.ndarray:
    """يُرجع القيم المعيارية (Z-scores) للمصفوفة المُدخلة."""
    return (data - data.mean()) / data.std()
```

١٠.٢ قابلية القراءة والصيانة

الكود الاحترافي في NumPy يوازن بين الإيجاز (الذي تشجعه Vectorization) والوضوح. سلسلة عمليات متجهة طويلة جدًا في سطر واحد قد تكون سريعة لكنها صعبة القراءة؛ التقسيم إلى متغيرات وسيطة ذات أسماء معبّرة غالبًا أفضل حتى لو كلّف سطرًا إضافيًا.

```
# أقل وضوحًا رغم كونه صحيحًا
result = ((data - data.mean(axis=0)) / data.std(axis=0))[mask].sum(axis=1)

# أوضح وأسهل صيانة
centered = data - data.mean(axis=0)
standardized = centered / data.std(axis=0)
filtered = standardized[mask]
result = filtered.sum(axis=1)
```

١٠.٣ هيكلة المشروع

```
src/
├── data_loading.py      # قراءة الملفات الخام فقط
├── preprocessing.py     # كل عمليات NumPy (تنظيف، تطبيع، تحويل)
├── features.py          # هندسة الخصائص العددية
└── utils/
    └── validation.py    # دوال assert للتحقق من الأشكال والأنواع
```

١٠.٤ الاختبار (Testing)

اختبار كود NumPy له خصوصية: المقارنة المباشرة a == b غير كافية للأعداد العشرية بسبب أخطاء التقريب، لذلك توفّر NumPy دوال مخصصة للمقارنة التقريبية.

```
import numpy as np
import numpy.testing as npt

def test_normalize():
```

```
    data = np.array([1.0, 2.0, 3.0])
    result = normalize(data)
    expected = np.array([0.0, 0.5, 1.0])
    npt.assert_allclose(result, expected, atol=1e-8)   # مقارنة تقريبية آمنة
```

استخدام npt.assert_allclose بدل == المباشرة هو من أهم ممارسات كتابة اختبارات موثوقة لكود عددي، لأن العمليات على الأعداد العشرية (float) نادرًا ما تنتج قيمًا مطابقة تمامًا بت لبت.

١٠.٥ التوثيق (Documentation)

المعيار الأكثر شيوعًا في المشاريع العلمية ببايثون هو أسلوب NumPy Docstring (المعروف أيضًا باسم numpydoc)، والذي يوثّق المعاملات والقيمة المُرجَعة بصيغة موحّدة يمكن لأدوات مثل Sphinx توليد توثيق HTML منها تلقائيًا.

```
def moving_average(data: np.ndarray, window: int) -> np.ndarray:
    """
    يحسب المتوسط المتحرك لمصفوفة أحادية البُعد.

    Parameters
    ----------
    data : np.ndarray
        مصفوفة القيم الأصلية أحادية البُعد.
    window : int
        حجم النافذة المتحركة.

    Returns
    -------
    np.ndarray
        مصفوفة المتوسطات المتحركة، أقصر من data بمقدار (window - 1).
    """
    ...
```

أفضل الممارسات

- قسّم سلاسل العمليات المتجهة الطويلة إلى متغيرات وسيطة واضحة الاسم.
- استخدم npt.assert_allclose بدل == للمقارنات في الاختبارات.
- وثّق الدوال بأسلوب numpydoc القياسي.
- افصل تحميل البيانات عن معالجتها عن عرضها في وحدات منفصلة.

الفصل الحادي عشر: مشاريع مصغّرة

مشاريع كاملة تعتمد فقط على NumPy، دون أي خوارزميات تعلّم آلي، بهدف تطبيق كل مفاهيم الأجزاء السابقة معًا.

مشروع ١: محاكاة نرد وحساب احتمالات تجريبية

```
import numpy as np

rng = np.random.default_rng(seed=7)
rolls = rng.integers(1, 7, size=100_000)   # محاكاة 100,000 رمية نرد

values, counts = np.unique(rolls, return_counts=True)
probabilities = counts / rolls.size

for v, prob in zip(values, probabilities):
    print(f"الرقم {v}: احتمال تجريبي = {prob:.4f}")
```

يطبّق هذا المشروع: Generator الحديث للأرقام العشوائية، np.unique مع return_counts، والعمليات المتجهة لحساب الاحتمالات دفعة واحدة.

مشروع ٢: ضغط صورة رمادية بسيط عبر تقليل الدقة (Downsampling)

```
image = np.random.randint(0, 256, (200, 200), dtype=np.uint8)

def downsample(img: np.ndarray, factor: int) -> np.ndarray:
    h, w = img.shape
    h2, w2 = h // factor, w // factor
    cropped = img[: h2 * factor, : w2 * factor]
    reshaped = cropped.reshape(h2, factor, w2, factor)
    return reshaped.mean(axis=(1, 3)).astype(np.uint8)

small = downsample(image, 4)
print(image.shape, '->', small.shape)   # (200, 200) -> (50, 50)
```

يطبّق هذا المشروع: إعادة التشكيل متعدد الأبعاد وaxis tuple في .mean()، وهو أسلوب متقدم يوضّح قوة التفكير بالمصفوفة كبنية هندسية متعددة الأبعاد بدل مجرد جدول ثنائي البُعد.

مشروع ٣: محرك توصية بسيط بالتشابه الإقليدي

```
users_ratings = np.array([
    [5, 3, 0, 1],
    [4, 0, 0, 1],
    [1, 1, 0, 5],
    [1, 0, 0, 4],
    [0, 1, 5, 4],
])   # 5 منتجات 4× مستخدمين

target_user = users_ratings[0]
distances = np.sqrt(((users_ratings - target_user) ** 2).sum(axis=1))
distances[0] = np.inf   # استبعاد المستخدم نفسه من الترشيح
```

```
most_similar = np.argmin(distances)
print(f"أقرب مستخدم للمستخدم 0 هو المستخدم {most_similar}")
```

يطبّق هذا المشروع: Broadcasting بين متجه وصف مصفوفة كاملة، axis=1 للجمع لكل مستخدم، وargmin لإيجاد أقرب تطابق — وهو مثال واقعي على استخدام NumPy وحدها كأساس حسابي قبل الانتقال لمكتبات تعلّم آلي متخصصة.

تمارين عملية

١. عدّل مشروع محاكاة النرد ليحاكي رمي نردين معًا وحساب احتمال كل مجموع ممكن من ٢ إلى ١٢.

٢. طوّر دالة downsample لتدعم صورًا ملوّنة بشكل (H, W, 3) بدل الرمادية فقط.

٣. أضف لمحرك التوصية إمكانية إرجاع أقرب ٣ مستخدمين بدل مستخدم واحد فقط باستخدام np.argsort.

حلول مقترحة (تلميحات)

١. rolls = rng.integers(1,7,(100000,2)).sum(axis=1) ثم np.unique كما في المثال.

٢. أضف بُعد القنوات للـ reshape وطبّق mean على المحاور المكانية فقط مع الإبقاء على بُعد القناة.

٣. استخدم np.argsort(distances)[:3] بعد استبعاد المستخدم نفسه.

مراجع وروابط رسمية

- التوثيق الرسمي: https://numpy.org/doc/stable/
- دليل الأداء الرسمي: https://numpy.org/doc/stable/user/basics.performance.html
- دليل numpydoc للتوثيق: https://numpydoc.readthedocs.io/
- الكود المصدري على GitHub: https://github.com/numpy/numpy

خاتمة NumPy

بهذا نكون قد أتممنا مرجعًا احترافيًا كاملاً لمكتبة NumPy عبر أحد عشر فصلاً: من التاريخ والفلسفة، مرورًا بالمفاهيم الجوهرية والدوال الأساسية، وصولاً إلى الأمثلة العملية والأداء والتصحيح وأفضل الممارسات والمشاريع المصغّرة. المكتبة التالية في هذه السلسلة هي Pandas، التي تُبنى مباشرة فوق كل ما تعلمته هنا.